

```
1 package com.distributed.scheduler;
2
3 import com.distributed.scheduler.common.enums.
    ExecutionStatus;
4 import com.distributed.scheduler.common.enums.
    JobStatus;
5 import com.distributed.scheduler.execution.dto.
    JobExecutionResponse;
6 import com.distributed.scheduler.execution.service.
    ExecutionService;
7 import com.distributed.scheduler.job.dto.
    CreateJobRequest;
8 import com.distributed.scheduler.job.dto.JobResponse;
9 import com.distributed.scheduler.job.entity.JobEntity
    ;
10 import com.distributed.scheduler.job.repository.
    JobRepository;
11 import com.distributed.scheduler.job.service.
    JobService;
12 import com.distributed.scheduler.project.dto.
    CreateProjectRequest;
13 import com.distributed.scheduler.project.dto.
    ProjectResponse;
14 import com.distributed.scheduler.project.service.
    ProjectService;
15 import com.distributed.scheduler.queue.dto.
    CreateQueueRequest;
16 import com.distributed.scheduler.queue.dto.
    QueueResponse;
17 import com.distributed.scheduler.queue.service.
    QueueService;
18 import com.distributed.scheduler.scheduler.service.
    ScheduledJobEligibilityService;
19 import org.junit.jupiter.api.DisplayName;
20 import org.junit.jupiter.api.Test;
21 import org.springframework.beans.factory.annotation.
    Autowired;
22 import org.springframework.boot.test.context.
    SpringBootTest;
23
24 import java.time.Instant;
```

```
25 import java.time.temporal.ChronoUnit;
26 import java.util.ArrayList;
27 import java.util.List;
28 import java.util.Map;
29 import java.util.UUID;
30 import java.util.concurrent.TimeUnit;
31
32 import static org.assertj.core.api.Assertions.
    assertThat;
33
34 @SpringBootTest
35 class DistributedJobSchedulerApplicationTests {
36
37     @Autowired
38     private ProjectService projectService;
39
40     @Autowired
41     private QueueService queueService;
42
43     @Autowired
44     private JobService jobService;
45
46     @Autowired
47     private JobRepository jobRepository;
48
49     @Autowired
50     private ExecutionService executionService;
51
52     @Autowired
53     private ScheduledJobEligibilityService
        scheduledJobEligibilityService;
54
55     @Test
56     @DisplayName("End-to-End Test: Create Project ->
        Create Queue -> Submit SEND_EMAIL Job -> Worker
        Claims & Executes -> COMPLETED")
57     void testEndToEndSendEmailJobExecutionFlow()
        throws Exception {
58         // 1. Create Project
59         String projectName = "Alpha-Project-" + UUID.
            randomUUID().toString().substring(0, 6);
```

```

60         ProjectResponse project = projectService.
            createProject(CreateProjectRequest.builder()
61                 .name(projectName)
62                 .description("Production test
project for background jobs")
63                 .build());
64
65         assertThat(project).isNotNull();
66         assertThat(project.getId()).isNotNull();
67         assertThat(project.getApiKey()).startsWith("
djs_live_");
68
69         // 2. Create Queue
70         QueueResponse queue = queueService.
            createQueue(CreateQueueRequest.builder()
71                 .projectId(project.getId())
72                 .name("email-notifications")
73                 .priority(10)
74                 .concurrencyLimit(5)
75                 .build());
76
77         assertThat(queue).isNotNull();
78         assertThat(queue.getName()).isEqualTo("email
-notifications");
79         assertThat(queue.getConcurrencyLimit()).
            isEqualTo(5);
80
81         // 3. Create SEND_EMAIL Job
82         Map<String, Object> emailPayload = Map.of(
83             "to", "lead-developer@company.org",
84             "subject", "System Alert:
Verification Completed",
85             "body", "All distributed scheduler
integration tests passed."
86         );
87
88         JobResponse submittedJob = jobService.
            createJob(CreateJobRequest.builder()
89                 .projectId(project.getId())
90                 .queueId(queue.getId())
91                 .jobType("SEND_EMAIL")

```

```
92         .payload(emailPayload)
93         .priority(10)
94         .build());
95
96     assertThat(submittedJob).isNotNull();
97     assertThat(submittedJob.getStatus()).
    isEqualTo(JobStatus.QUEUED);
98     assertThat(submittedJob.getJobType()).
    isEqualTo("SEND_EMAIL");
99
100    // 4. Wait for Worker to poll, atomically
    claim, and execute the job to COMPLETED
101    UUID jobId = submittedJob.getId();
102    boolean completed = false;
103    long startTime = System.currentTimeMillis();
104
105    while (System.currentTimeMillis() -
    startTime < 10000) {
106        JobResponse currentJob = jobService.
    getJobById(jobId);
107        if (currentJob.getStatus() == JobStatus.
    COMPLETED) {
108            completed = true;
109            break;
110        }
111        TimeUnit.MILLISECONDS.sleep(200);
112    }
113
114    assertThat(completed)
115        .withFailMessage("Job did not
    transition to COMPLETED within timeout")
116        .isTrue();
117
118    // 5. Verify Execution Record & Logs
119    List<JobExecutionResponse> executions =
    executionService.getExecutionsForJob(jobId);
120    assertThat(executions).isNotEmpty();
121    JobExecutionResponse exec = executions.get(0
    );
122    assertThat(exec.getStatus()).isEqualTo(
    ExecutionStatus.COMPLETED);
```

```
123         assertThat(exec.getAttemptNumber()).
            isEqualTo(1);
124         assertThat(exec.getWorkerId()).isNotBlank();
125         assertThat(exec.getDurationMs()).
            isGreaterThan(0);
126         assertThat(exec.getResultOutput()).isNotNull
            ();
127         assertThat(exec.getResultOutput().get("
            recipient")).isEqualTo("lead-developer@company.org"
            );
128
129         // 6. Verify Execution Logs
130         assertThat(exec.getLogs()).isNotEmpty();
131         boolean foundLog = exec.getLogs().stream()
132             .anyMatch(l -> l.getMessage().
                contains("SMTP transmission"));
133         assertThat(foundLog).isTrue();
134     }
135
136     @Test
137     @DisplayName("Concurrent Batch Execution: Submit
        multiple jobs and verify all are executed safely
        without duplicates")
138     void testConcurrentBatchExecution() throws
        Exception {
139         ProjectResponse project = projectService.
            createProject(CreateProjectRequest.builder()
140                 .name("Batch-Project-" + UUID.
                    randomUUID().toString().substring(0, 6))
141                 .description("Batch processing test"
                )
142                 .build());
143
144         QueueResponse queue = queueService.
            createQueue(CreateQueueRequest.builder()
145                 .projectId(project.getId())
146                 .name("batch-email-queue")
147                 .priority(5)
148                 .concurrencyLimit(10)
149                 .build());
150
```

```
151         int jobCount = 5;
152         List<UUID> jobIds = new ArrayList<>();
153
154         for (int i = 1; i <= jobCount; i++) {
155             JobResponse job = jobService.createJob(
156                 CreateJobRequest.builder()
157                     .projectId(project.getId())
158                     .queueId(queue.getId())
159                     .jobType("SEND_EMAIL")
160                     .payload(Map.of("to", "batch-
161                         user-" + i + "@example.com", "subject", "Batch Alert
162                         #" + i))
163                     .priority(i)
164                     .build());
165             jobIds.add(job.getId());
166         }
167
168         long startTime = System.currentTimeMillis();
169         boolean allCompleted = false;
170
171         while (System.currentTimeMillis() -
172             startTime < 15000) {
173             long completedCount = jobIds.stream()
174                 .map(jobService::getJobById)
175                 .filter(j -> j.getStatus() ==
176                     JobStatus.COMPLETED)
177                 .count();
178
179             if (completedCount == jobCount) {
180                 allCompleted = true;
181                 break;
182             }
183             TimeUnit.MILLISECONDS.sleep(300);
184         }
185
186         assertThat(allCompleted)
187             .withFailMessage("Not all batch jobs
188                 completed within 15 seconds")
189             .isTrue();
190
191         // Verify each job had exactly 1 execution
```

```

185 attempt
186     for (UUID jId : jobIds) {
187         List<JobExecutionResponse> execs =
executionService.getExecutionsForJob(jId);
188         assertThat(execs).hasSize(1);
189         assertThat(execs.get(0).getStatus()).
isEqualTo(ExecutionStatus.COMPLETED);
190     }
191 }
192
193 @Test
194 @DisplayName("Scheduler Module: Scheduled/
Delayed job is created as SCHEDULED and promoted to
QUEUED when due")
195 void testScheduledJobPromotionBySchedulerModule
() throws Exception {
196     ProjectResponse project = projectService.
createProject(CreateProjectRequest.builder()
197         .name("Scheduler-Project-" + UUID.
randomUUID().toString().substring(0, 6))
198         .description("Delayed test")
199         .build());
200
201     QueueResponse queue = queueService.
createQueue(CreateQueueRequest.builder()
202         .projectId(project.getId())
203         .name("scheduled-queue")
204         .build());
205
206     // 1. Submit job scheduled in the future ->
created as SCHEDULED
207     JobResponse job = jobService.createJob(
CreateJobRequest.builder()
208         .projectId(project.getId())
209         .queueId(queue.getId())
210         .jobType("SEND_EMAIL")
211         .payload(Map.of("to", "delayed@
example.com")))
212         .scheduledAt(Instant.now().plus(2,
ChronoUnit.HOURS))
213         .build());

```

```
214
215         assertThat(job.getStatus()).isEqualTo(
    JobStatus.SCHEDULED);
216
217         // 2. Simulate time reaching execution time
    (update scheduledAt to now)
218         JobEntity entity = jobRepository.findById(
    job.getId()).orElseThrow();
219         entity.setScheduledAt(Instant.now().minus(1
    , ChronoUnit.SECONDS));
220         jobRepository.save(entity);
221
222         // 3. Scheduler module promotes due jobs
    scheduledJobEligibilityService.
223         promoteDueScheduledJobs();
224
225         // 4. Job is now QUEUED and subsequently
    picked up by worker to COMPLETED
226         long startTime = System.currentTimeMillis();
227         boolean completed = false;
228
229         while (System.currentTimeMillis() -
    startTime < 10000) {
230             JobResponse currentJob = jobService.
    getJobById(job.getId());
231             if (currentJob.getStatus() == JobStatus.
    COMPLETED) {
232                 completed = true;
233                 break;
234             }
235             TimeUnit.MILLISECONDS.sleep(200);
236         }
237
238         assertThat(completed).isTrue();
239     }
240 }
241
```